Compte Rendu d'Atelier : Améliorer la conception d'AutoLoc avec GoF et SOLID

Module : Architecture des Systèmes d'Information (ASI 4CCE13) — ESPRIT
Étudiant : Amine Mezni | **Email :** amine.mezni@esprit.tn | **Date :** 28 Octobre 2026

---

1. Partie A — Identifier les problèmes de conception

Q1. Différentes responsabilités prises en charge par VehiculeService :

Dans le code initial fourni, la classe cumule 4 responsabilités hétérogènes :

- **Orchestration métier CRUD** : coordination des opérations sur les véhicules (ajout, calcul).
- **Validation des données** : vérification des contraintes d'intégrité (tarif positif, nombre de jours).
- **Journalisation (Logging)** : traçabilité via System.out.println("Ajout du véhicule").
- **Calcul financier / Algorithmique tarifaire** : formule monétaire dans calculerTarif.

Q2. Pourquoi ces responsabilités doivent évoluer indépendamment :

La validation dépend des règles métier et légales (ex: règles de tarification minimale ou immatriculation tunisienne). La journalisation est une préoccupation purement technique et d'infrastructure (remplacement par Logback, envoi vers ELK). La gestion du véhicule évolue selon les besoins des agences. Les coupler dans une seule classe génère un risque élevé de régression.

Q3 & Q4. Principe SOLID et répartition proposée :

Le principe fondamental est le **Single Responsibility Principle (SRP)** : une classe ne doit avoir qu'une seule raison de changer.

Composant	Responsabilité unique
VehiculeService	Coordonne les opérations métier et orchestre les flux entre composants.
VehiculeValidator	Encapsule toutes les vérifications et assertions d'intégrité métier.
JournalisationService	Centralise et formate les traces et audits applicatifs.
TarificationService	Prend en charge le calcul financier et la tarification.

2. Partie B — Appliquer SRP et DIP

B.1. Refactorisation SRP de la méthode ajouter() :

```
@Service @RequiredArgsConstructor public class VehiculeServiceImpl implements IVehiculeService {
private final VehiculeRepository vehiculeRepository; private final VehiculeValidator
vehiculeValidator; // SRP : Validation private final JournalisationService journalisationService;
// SRP : Logs @Override public Vehicule ajouterVehicule(Vehicule vehicule) {
vehiculeValidator.valider(vehicule); // Étape 1 : Valider
journalisationService.journaliserAction("AJOUT", "Vehicule", vehicule.getImmatriculation());
return vehiculeRepository.save(vehicule); // Étape 2 : Persister } }
```

B.2. Principe d'Inversion des Dépendances (DIP) :

1. **Rôle de VehiculeRepository :** C'est une interface d'abstraction qui fournit le contrat d'accès aux données sans exposer les détails techniques (SQL/Hibernate).

2. **Pourquoi dépendre d'un contrat :** Pour garantir un couplage faible, permettre les tests unitaires via des mocks (`Mockito`) et autoriser le changement de technologie sans impacter le service.

3. **Mécanisme d'injection Spring :** Spring Data crée un proxy dynamique au runtime et l'injecte par constructeur (`@RequiredArgsConstructor`) dans `VehiculeServiceImpl`.

4. **Lien entre DIP et DI :** Le **DIP** est le principe d'architecture conceptuel (*dépendre d'abstractions*). La **DI** est le mécanisme technique concret par lequel le framework instancie et fournit l'abstraction.

3. Partie C — GoF Strategy et principe OCP

Architecture du patron Strategy implémentée :

- **Interface :** StrategieTarification avec la méthode double calculer(double tarifJournalier, int jours).
- **Stratégies concrètes :** TarificationStandard (100%), TarificationWeekend (-10%), TarificationLongueDuree (-20%), et TarificationEntreprise (-15%).
- **Contexte :** TarificationService reçoit automatiquement une Map<String, StrategieTarification> injectée par Spring.

```
@Service @RequiredArgsConstructor public class TarificationService { private final Map strategies; public double calculerPrix(String typePolitique, double tarifJournalier, int jours) { StrategieTarification strategie = strategies.get(typePolitique.toUpperCase()); if (strategie == null) { throw new IllegalArgumentException("Type de tarif inconnu : " + typePolitique); } return strategie.calculer(tarifJournalier, jours); } }
```

Réponses aux questions :

1. **Problème résolu :** Élimination des cascades conditionnelles (`switch` ou `if-else`) en encapsulant chaque formule dans une classe interchangeable.

2. **Intérêt d'isoler chaque formule :** Séparation des règles (SRP), facilité des tests unitaires et étanchéité aux régressions.

3. **Application d'OCP (Open/Closed Principle) :** Le système est *ouvert à l'extension* (l'ajout de `TarificationEntreprise` s'est fait simplement en créant la classe avec `@Component("ENTREPRISE")`) et *fermé à la modification* (`TarificationService` est resté strictement inchangé).

4. **Cas de politique inexistante :** Lever une IllegalArgumentException explicite listant les politiques valides disponibles.

4. Partie D — Reconnaître Factory, ISP et LSP

Situation 1 — Factory (Génération de factures PDF / Excel) :

- **Problème résolu :** Masque la complexité d'instanciation des bibliothèques de génération (iText pour PDF, Apache POI pour Excel).
- **Méthode exposée :** public FactureDocument genererFacture(TypeFormat format, Contrat contrat).
- **Avantage :** Point unique de création, couplage lâche, conformité OCP lors de l'ajout d'un format (ex: Factur-X / XML).

Situation 2 — ISP (Interface Segregation Principle) :

L'interface GestionVehicule est une 'interface grasse' violant l'ISP. Aucun client ne devrait être contraint de dépendre de méthodes qu'il n'utilise pas. **Solution :** Découper en 4 interfaces spécialisées :

- VehiculeLectureService : consulter(Long id)
- VehiculeModificationService : ajouter(), modifier(), supprimer()
- VehiculeRapportService : exporterRapport()
- VehiculeNotificationService : envoyerNotification()

Situation 3 — LSP (Liskov Substitution Principle) :

Lever UnsupportedOperationException brise le contrat comportemental promis par la classe mère. L'appelant s'attend à une exécution normale et subit une erreur au runtime. **Solution :** Privilégier la composition à l'héritage (*'Favor composition over inheritance'*), ou réviser la hiérarchie pour que la sous-classe n'hérite que des contrats qu'elle peut honorer.

5. Synthèse globale des principes SOLID et GoF

Principe / Patron	Définition	Implémentation dans AutoLoc
SRP	Une seule responsabilité par classe.	Séparation de VehiculeService, VehiculeValidator, JournalisationService.
OCP	Ouvert à l'extension, fermé à la modification.	Ajout de TarificationEntreprise sans modifier TarificationService.
LSP	Les sous-types doivent être substituables.	Respect strict des contrats d'interfaces sans lever d'exceptions inattendues.
ISP	Préférer plusieurs interfaces spécifiques.	Contrats ciblés (StrategieTarification, JournalisationService).
DIP	Dépendre d'abstractions, pas d'implémentations.	Injection de VehiculeRepository et StrategieTarification par Spring.
GoF Strategy	Famille d'algorithmes interchangeables.	Politiques tarifaires (STANDARD, WEEKEND, LONGUE_DUREE, ENTREPRISE).
GoF Factory	Centralise la création d'objets complexes.	Conception de la fabrique de factures (PDF / Excel).